```python
def solve_n_queens(N):
    board = [-1] * N  # index = row, value = column
    solutions = []

    def is_safe(current_row, test_col):
        for prev_row in range(current_row):
            prev_col = board[prev_row]

            if prev_col == test_col:
                return False

            if abs(prev_row - current_row) == abs(prev_col - test_col):
                return False

        return True

    def backtrack(row):
        if row == N:
            solutions.append(list(board))
            return

        for col in range(N):
            if is_safe(row, col):
                board[row] = col
                backtrack(row + 1)

    print(f"--- Solving {N}-Queens using Backtracking ---")
    backtrack(0)
    return solutions

def print_board(solution, N):
    for col in solution:
        row_str = ["."] * N
        row_str[col] = "Q"
        print(" ".join(row_str))
    print("-" * (N * 2))

N_size = 4
found_solutions = solve_n_queens(N_size)

print(f"Found {len(found_solutions)} valid solutions for a {N_size}x{N_size} board:\n")
for idx, sol in enumerate(found_solutions):
    print(f"Solution {idx + 1}: {sol}")
    print_board(sol, N_size)
```